JARVIS Workbench
v1 Execution Plan

Every Step broken into tickets. Each ticket has a start state, a checklist, a finish line, and the commit that closes it.

0 → 1 → 2a → **2b** → R → 2c → 3 → 4 → 5 → 6 → 7 → 8

HARDWARE
CYD ESP32-2432S028R (2-USB)

SCOPE
9 Steps · 48 tickets remaining

OWNER
Antwaun · solo developer

How this plan works

READ ONCE, FOLLOW EVERY SESSION

This is a solo version of how professional teams run work. A team tracks work as tickets on a board, and each ticket has a clear entry and exit. You are the whole team, so the rituals are short, but the rules are the same.

Step (epic)	A large chunk of work. One branch, one pull request, one merge.
Ticket	One session-sized micro task. Usually ends in one commit.
Start state	What must already be true before you begin. Professionals call it the *Definition of Ready*. If it isn't true, go back to the previous ticket.
Done when	The only finish line. Professionals call it the *Definition of Done*. "Mostly works" is not done.
WIP limit = 1	WIP means Work In Progress. Only one ticket open at a time. Finish or park it before starting another.
Timebox	Stuck for 30 minutes? Stop. Write what you tried and what you saw in the devlog, then research or ask.
Smoke test	A quick "does it still turn on" check: screen shows its content, LED heartbeat blinks, serial shows the init lines, and touch responds once it exists.

SESSION START · 5 MIN

Like a team stand-up meeting

- [ ] `git status`: working tree clean?
- [ ] On the right branch? `git pull` if you work across machines.
- [ ] Open the tracker. Find the next unchecked ticket.
- [ ] Read its **Start state**. Is it true?
- [ ] Read the checklist. Write a one-line prediction of the result.
- [ ] Set a timer for your session length.

SESSION END · 5 MIN

Leave a clean desk for future-you

- [ ] Ticket done? Run the global Definition of Done, then commit.
- [ ] Not done? Commit what builds and write where you stopped.
- [ ] `git push` the branch (it is your off-site backup).
- [ ] Write the devlog entry (template below).
- [ ] Tick the box and date it in the tracker.
- [ ] Tell Claude "session done" to get your notes and resources.

Devlog entry · append to `docs/devlog.md` at session end

```
## 2026-10-12 · Ticket 2b.3
Did:      started XPT2046 on VSPI, "Touch init done" prints
Learned:  touch needs its own SPIClass because it is on a different bus
Blockers: none
Next:     2b.4, print raw x/y/z with printf
```

Standards

APPLY TO EVERY TICKET

GLOBAL DEFINITION OF DONE

Every ticket, no exceptions

- [ ] Builds with zero errors and no new warnings.
- [ ] Smoke test passes.
- [ ] Ticket's own **Done when** is true.
- [ ] No magic numbers: pins, timings and thresholds are `constexpr`.
- [ ] No `delay()` in `loop()`.
- [ ] Comments match the code.
- [ ] Docs updated if a pin, decision or measurement changed.
- [ ] Committed with a conventional commit message.

BLOCKER PROTOCOL

When something won't work

- [ ] Re-read the symptom. What exactly did you see?
- [ ] Compare it with your prediction.
- [ ] Change one thing at a time, and write each change down.
- [ ] Check the boring stuff: power, wiring, baud rate, correct env.
- [ ] At 30 minutes, log it in the devlog and ask.

Writing the blocker down often solves it. Engineers call this rubber-duck debugging.

Commit types

TYPE	USE IT FOR	EXAMPLE
`feat`	A new ability	`feat(touch): map raw touch to pixels`
`fix`	Fixing a bug	`fix(rfid): stop double-reads on held card`
`refactor`	Restructure code, same behavior	`refactor(display): extract display module`
`build`	platformio.ini, libraries, flags	`build(ui): add LVGL pinned to 9.x.y`
`docs`	Anything in docs/ or hardware/	`docs(wiring): add touch pin map`
`test`	Bench tests, scanners, soak runs	`test(opto): bench-verify PC817`
`style`	Visual tuning, formatting	`style(hud): tune colors for mirror`
`chore`	Housekeeping (.gitignore, templates)	`chore(config): add secrets template`

SHIP CHECKLIST · THE LAST TICKET OF EVERY STEP

Branch to merged

- [ ] Run the full smoke test one last time (screen, touch if present, LED heartbeat, serial).
- [ ] `git status` is clean and `git log --oneline` reads like a clear story.
- [ ] `git push` the branch.
- [ ] Open the PR (pull request): **What** changed / **How tested** / **Notes**.
- [ ] Review your own diff on GitHub in the *Files changed* tab, like a teammate would.
- [ ] Merge → `git checkout main` → `git pull` → delete the branch (local and on GitHub).
- [ ] Tick the Step in the Progress Tracker and write the devlog entry.

Progress tracker

TICK AND DATE AS YOU GO

Completed — main

- [x] 0 Repo and requirements _____
- [x] 1 Board bring-up _____
- [x] 2a Display bring-up _____

Step 2b · Touch bring-up — feature/touch-bringup

- [ ] 2b.1 Document touch pins _____
- [ ] 2b.2 Add the touch library _____
- [ ] 2b.3 Start the touch controller _____
- [ ] 2b.4 Print raw touch values _____
- [ ] 2b.5 Record the corners (calibration data) _____
- [ ] 2b.6 Map raw values to pixels _____
- [ ] 2b.7 Pressure filter _____
- [ ] 2b.8 Ship it (PR + merge) _____

Step R · Refactor into modules — refactor/modules

- [ ] R.1 Display module _____
- [ ] R.2 Touch module _____
- [ ] R.3 LED module (your first HAL) _____
- [ ] R.4 Thin main _____

Step 2c · LVGL user interface — feature/lvgl

- [ ] 2c.1 Add LVGL and log the decision _____
- [ ] 2c.2 Config file (lv_conf.h) _____
- [ ] 2c.3 Connect LVGL to the screen _____
- [ ] 2c.4 Give LVGL a heartbeat _____
- [ ] 2c.5 Connect touch to LVGL _____
- [ ] 2c.6 Wake Up and Go Back to Sleep buttons _____

Step 3 · PN532 RFID reader — feature/rfid

- [ ] 3.1 Learn the part _____
- [ ] 3.2 Wire it _____
- [ ] 3.3 Find it on the bus _____
- [ ] 3.4 Talk to it _____
- [ ] 3.5 Read cards without blocking _____
- [ ] 3.6 Project cards _____

Step 4 · Optocoupler bench test — feature/opto

- [ ] 4.1 Pick a drive pin _____
- [ ] 4.2 Do the resistor math _____
- [ ] 4.3 Breadboard test _____
- [ ] 4.4 Non-blocking press function _____
- [ ] 4.5 Connect to the real PC _____

Step 5 · PC sense — feature/pc-sense

- [ ] 5.1 Choose a method _____
- [ ] 5.2 Build the sense circuit _____
- [ ] 5.3 pcIsOn() firmware _____

Step 6 · State machine — feature/state-machine

- [ ] 6.1 Draw it before coding it _____
- [ ] 6.2 Skeleton _____
- [ ] 6.3 Card wakes the screen _____
- [ ] 6.4 Buttons do real things _____
- [ ] 6.5 Edge cases _____

Step 7 · HUD — feature/hud

- [ ] 7.1 Secrets file _____
- [ ] 7.2 Real time over the network _____
- [ ] 7.3 Layout sketch _____
- [ ] 7.4 Clock _____
- [ ] 7.5 Project and task _____
- [ ] 7.6 Mirror test _____

Step 8 · Install and release — feature/install

- [ ] 8.1 Plan the mount _____
- [ ] 8.2 Power supply _____
- [ ] 8.3 Install and route cables _____
- [ ] 8.4 24-hour soak test _____
- [ ] 8.5 Release v1.0.0 _____

Step 2b Touch bring-up IN PROGRESS

`feature/touch-bringup`

GOAL A finger touch reports accurate screen coordinates.

2b.1 Document touch pins

Done _____

START STATE On `feature/touch-bringup`. `docs/wiring.md` exists with the template.

CHECKLIST

- [ ] Open `PINS.md` in the witnessmenow CYD repo → *Touch Screen* section.
- [ ] Fill in the GPIO numbers for CLK, MOSI, MISO, CS and IRQ.
- [ ] In Notes, mark which pins are **input-only** and write one line on why that is fine for them.
- [ ] Add the five touch pins to the *Reserved* table.
- [ ] Update the *Last verified* date.
- [ ] `git diff` → review → commit.

DONE WHEN **Touch table complete, input-only pins noted, committed.**

COMMIT `docs(wiring): add XPT2046 touch pin map`

2b.2 Add the touch library

Done _____

START STATE 2b.1 committed. The project builds with no errors.

CHECKLIST

- [ ] Find `XPT2046_Touchscreen` by Paul Stoffregen in the PlatformIO registry.
- [ ] Add it to `lib_deps` with a version pin.
- [ ] Build. Zero errors.
- [ ] Confirm the library folder appeared in `.pio/libdeps/`.
- [ ] Skim `XPT2046_Touchscreen.h`: find `touched()`, `getPoint()`, and the field types inside `TS_Point`.
- [ ] Commit `platformio.ini`.

DONE WHEN **Builds clean with the library installed.**

COMMIT `build(touch): add XPT2046_Touchscreen library`

2b.3 Start the touch controller

Done _____

START STATE: 2b.2 committed. Screen still shows "Hello JARVIS".

CHECKLIST:

- [ ] Write the TODO comments first (TODO 1–8 from the skeleton).
- [ ] Define the five touch pins as `constexpr`.
- [ ] Create a global `SPIClass` object on the VSPI bus.
- [ ] Create a global touchscreen object (needs CS and IRQ).
- [ ] In `setup()`: start SPI with the 4 pins → start touch with your SPI object → set touch rotation to match the display (3).
- [ ] Print "Touch init done" as the last touch line in `setup()`.
- [ ] Write down your prediction of what you will see.
- [ ] Build → upload → open the serial monitor. Compare with your prediction.

DONE WHEN: **Serial shows "Display init done" and "Touch init done". Text still on screen. LED still blinks.**

COMMIT: `feat(touch): initialize XPT2046 on VSPI bus`

2b.4 Print raw touch values

Done _____

START STATE: 2b.3 passes.

CHECKLIST:

- [ ] In `loop()`, check `touched()`.
- [ ] If touched: `getPoint()` → `printf` x, y, z on one line with the right format specifiers.
- [ ] Rate-limit the print to about every 100 ms with `millis()` (same pattern as the heartbeat).
- [ ] Flash. Touch the center and each edge.
- [ ] Check: no touch = no output.
- [ ] Check: LED still blinks at its normal rate (proves `loop()` is not blocked).

DONE WHEN: **Touches print values, no touch prints nothing, LED rate unchanged.**

COMMIT: `feat(touch): log raw touch coordinates to serial`

2b.5 Record the corners (calibration data)

Done _____

START STATE: 2b.4 passes.

CHECKLIST:

- [ ] Create `docs/calibration.md` with a table: Corner | Raw X | Raw Y | Z.
- [ ] Touch each corner 3 times with a fingernail or stylus. Record the typical value.
- [ ] Note which raw axis grows left→right and which grows top→bottom (they can be flipped or swapped).
- [ ] Write down the overall min and max for X and Y.
- [ ] Commit.

DONE WHEN: **Four corners plus min/max recorded and committed.**

COMMIT: `docs(calibration): record raw touch corner values`

2b.6 Map raw values to pixels

Done _____

START STATE `calibration.md` committed.

CHECKLIST

- [ ] Create `constexpr` min/max values from `calibration.md` (no magic numbers).
- [ ] `map()` raw X/Y to `0..tft.width()-1` and `0..tft.height()-1`.
- [ ] `constrain()` the result so it can never land off-screen.
- [ ] If an axis runs backward, swap its min/max in `map()`.
- [ ] Draw a small filled circle at the mapped point.
- [ ] Test all four corners and the center.

DONE WHEN **The dot lands under your finger (within about 5 px) at the corners and center.**

COMMIT `feat(touch): map raw touch to screen pixels`

2b.7 Pressure filter

Done _____

START STATE 2b.6 passes.

CHECKLIST

- [ ] Watch Z in serial: compare a light brush against a normal tap.
- [ ] Pick a threshold between them. Store it as `constexpr TOUCH_MIN_Z`.
- [ ] Ignore any point below the threshold.
- [ ] Test: 10 light brushes, 10 normal taps.
- [ ] Record the chosen value and why in `calibration.md`.

DONE WHEN **Light brushes are ignored, normal taps register every time.**

COMMIT `feat(touch): ignore low-pressure touches`

2b.8 Ship it (PR + merge)

Done _____

START STATE 2b.1–2b.7 done.

CHECKLIST

- [ ] Run the full smoke test one last time (screen, touch if present, LED heartbeat, serial).
- [ ] `git status` is clean and `git log --oneline` reads like a clear story.
- [ ] `git push` the branch.
- [ ] Open the PR (pull request): **What** changed / **How tested** / **Notes**.
- [ ] Review your own diff on GitHub in the *Files changed* tab, like a teammate would.
- [ ] Merge → `git checkout main` → `git pull` → delete the branch (local and on GitHub).
- [ ] Tick the Step in the Progress Tracker and write the devlog entry.

DONE WHEN **PR merged, local `main` pulled, branch deleted.**

COMMIT `(merge commit on GitHub)`

Step R Refactor into modules UP NEXT

refactor/modules

GOAL `main.cpp` becomes the manager. Each part is its own module. Behavior does not change.

R.1 Display module

Done _____

START STATE Fresh branch from updated `main`. Everything works.

CHECKLIST

- [ ] Create `include/display.h` with `#pragma once` (stops the file being included twice).
- [ ] Declare `displayInit()` and any display functions `main.cpp` uses.
- [ ] Create `src/display.cpp`. Move the `tft` object there as `static` (private to that file).
- [ ] Move the display setup code into `displayInit()`.
- [ ] In `main.cpp`: include `display.h` and call `displayInit()`.
- [ ] Build and smoke test: behavior identical.

DONE WHEN **Builds. Screen behaves exactly as before.**

COMMIT `refactor(display): extract display module`

R.2 Touch module

Done _____

START STATE R.1 committed.

CHECKLIST

- [ ] `touch.h`: declare `touchInit()` and `bool touchRead(int16_t *x, int16_t *y)`.
- [ ] `touch.cpp`: SPI and touchscreen objects are `static`. Mapping and pressure filter live inside.
- [ ] `touchRead()` returns true only for a valid touch and writes pixel coordinates through the pointers.
- [ ] Add `displayDrawDot(x, y)` to the display module, so `main.cpp` never touches `tft`.
- [ ] Replace the touch code in `main.cpp` with the new calls.
- [ ] Smoke test.

DONE WHEN **Builds. The dot still follows your finger.**

COMMIT `refactor(touch): extract touch module`

R.3 LED module (your first HAL)

Done _____

START STATE R.2 committed.

CHECKLIST

- [ ] `led.h`: `ledInit()`, `ledSet(bool on)`, `ledUpdate()`.
- [ ] Hide the active-LOW detail inside `led.cpp`. Callers just say on/off.
- [ ] Move the blink timing into `ledUpdate()`.
- [ ] Search `main.cpp`: zero `digitalWrite` or `pinMode` calls left.

DONE WHEN **No direct pin calls in `main.cpp`. LED blinks as before.**

COMMIT `refactor(led): add LED hardware abstraction module`

R.4 Thin main

Done _____

START STATE: R.3 committed.

CHECKLIST:

- [ ] `setup()` contains only `Serial.begin`, the boot line, and `xxxInit()` calls.
- [ ] `loop()` contains only `xxxUpdate()` calls.
- [ ] Move the serial heartbeat into its own function or module.
- [ ] Count the lines: under about 40.
- [ ] Smoke test, then run the Ship checklist (same as 2b.8).

DONE WHEN: **`main.cpp` reads like a table of contents. PR merged.**

COMMIT: `refactor(main): reduce main to init/update calls`

Step 2c LVGL user interface

`feature/lvgl`

GOAL A real UI library (LVGL, Light and Versatile Graphics Library) with tappable buttons.

2c.1 Add LVGL and log the decision

Done _____

START STATE Step R merged. Fresh branch from `main`.

CHECKLIST
- [ ] Read the LVGL Arduino getting-started docs.
- [ ] Pick an exact version. v8 and v9 code looks different, so every tutorial you follow must match your version.
- [ ] Add it to `lib_deps` pinned exactly (`@x.y.z`).
- [ ] Create `docs/decisions.md`. Entry: date, decision, reason, alternatives considered.
- [ ] Build.

DONE WHEN **Builds. Decision logged.**

COMMIT `build(ui): add LVGL pinned to exact version`

2c.2 Config file (lv_conf.h)

Done _____

START STATE 2c.1 committed.

CHECKLIST
- [ ] Copy `lv_conf_template.h` from the library into `include/lv_conf.h`.
- [ ] Change its top `#if 0` to `#if 1` (switches the file on).
- [ ] Set color depth to 16 (matches the screen's 16-bit color).
- [ ] Enable the font sizes you want.
- [ ] Point the build at the file (look up `LV_CONF_INCLUDE_SIMPLE`).
- [ ] Build.

DONE WHEN **Builds with no LVGL config warnings.**

COMMIT `build(ui): add lv_conf.h configuration`

2c.3 Connect LVGL to the screen

Done _____

START STATE 2c.2 committed.

CHECKLIST
- [ ] Read LVGL's *Display interface* page for your version.
- [ ] Allocate a draw buffer (start at 1/10 of the screen).
- [ ] Write the flush callback: hand the pixel area to TFT_eSPI (look up `setAddrWindow` and `pushColors`).
- [ ] Tell LVGL the flush is finished (the *flush ready* call for your version).
- [ ] Register the display with LVGL.
- [ ] Create one label: "Hello JARVIS".

DONE WHEN **The LVGL label shows on screen.**

COMMIT `feat(ui): connect LVGL to TFT display`

2c.4 Give LVGL a heartbeat

Done _____

START STATE: 2c.3 passes.

CHECKLIST:
- [ ] Give LVGL its time source (the *tick*), using `millis()`.
- [ ] Call `lv_timer_handler()` every loop.
- [ ] Confirm `loop()` has no `delay()`.
- [ ] Show uptime in seconds on a label, updated by an `lv_timer`, not by `loop()`.

DONE WHEN: **Label updates once per second. LED still blinks normally.**

COMMIT: `feat(ui): add LVGL tick and timer handler`

2c.5 Connect touch to LVGL

Done _____

START STATE: 2c.4 passes.

CHECKLIST:
- [ ] Read LVGL's *Input device* page.
- [ ] Write the read callback: call `touchRead()`, set the point and pressed/released state.
- [ ] Register it as a pointer input device.
- [ ] Add a test button that changes color when pressed.

DONE WHEN: **Pressing the button changes its color. Releasing restores it.**

COMMIT: `feat(ui): connect touch input to LVGL`

2c.6 Wake Up and Go Back to Sleep buttons

Done _____

START STATE: 2c.5 passes.

CHECKLIST:
- [ ] Create both buttons, laid out for landscape.
- [ ] Attach a callback to each for the *clicked* event.
- [ ] Each callback prints one line with `printf`.
- [ ] Tap each 10 times: exactly one line per tap.
- [ ] Smoke test, then run the Ship checklist.

DONE WHEN: **Each tap prints exactly one line. PR merged.**

COMMIT: `feat(ui): add wake and sleep buttons`

Step 3 PN532 RFID reader

`feature/rfid`

GOAL Tapping a card prints its project name without freezing the screen.

3.1 Learn the part

Done _____

START STATE: Step 2c merged. Fresh branch from `main`.

CHECKLIST:

- [ ] Find your PN532 board's docs and read the labels printed on the board.
- [ ] Find the mode switch table (I2C / SPI / HSU).
- [ ] With power **off**, set the switches to I2C (Inter-Integrated Circuit, the two-wire bus).
- [ ] Confirm the board accepts 3.3 V.
- [ ] Add a PN532 section to `wiring.md` with the switch positions.

DONE WHEN: **Mode and switch positions documented.**

COMMIT: `docs(wiring): add PN532 mode settings`

3.2 Wire it

Done _____

START STATE: 3.1 committed. Everything unpowered.

CHECKLIST:

- [ ] Identify the CN1 connector pins in `PINS.md`: GND, IO22, IO27, 3.3 V.
- [ ] Choose which pin is SDA (data) and which is SCL (clock). Record it.
- [ ] Wire 3.3 V, GND, SDA, SCL. Check every wire against `wiring.md` before power.
- [ ] Multimeter continuity check on each wire.
- [ ] Save a photo to `hardware/photos/`.

DONE WHEN: **Wired, verified, documented with a photo.**

COMMIT: `docs(hardware): add PN532 wiring and photo`

3.3 Find it on the bus

Done _____

START STATE: 3.2 committed.

CHECKLIST:

- [ ] Write a temporary I2C scanner function: start `Wire` on your SDA/SCL pins, try every address, print what answers.
- [ ] Expect `0x24`.
- [ ] Nothing found? Check switches, power, then swapped SDA/SCL.
- [ ] Remove the scanner or keep it behind a debug flag.

DONE WHEN: **Serial shows the PN532 at its address.**

COMMIT: `test(rfid): verify PN532 on I2C bus`

3.4 Talk to it

Done _____

START STATE: 3.3 passes.

CHECKLIST:

- [ ] Choose a PN532 library. Pin its version. Log it in `decisions.md`.
- [ ] Initialize over I2C.
- [ ] Read the chip's firmware version and print it at boot.
- [ ] If init fails, print a clear error and keep running. Never hang forever.

DONE WHEN: **Firmware version prints at boot. A disconnected reader gives a clear error.**

COMMIT: `feat(rfid): initialize PN532 and read firmware version`

3.5 Read cards without blocking

Done _____

START STATE: 3.4 passes.

CHECKLIST:

- [ ] Find out how long the library's read call can wait (its timeout).
- [ ] Use a short timeout, and only check every 100–200 ms with `millis()`.
- [ ] Print the card UID (unique ID) in hex.
- [ ] Holding a card prints once until it is removed.
- [ ] Check the LVGL buttons still respond while reading.

DONE WHEN: **Each tap prints once. UI stays responsive.**

COMMIT: `feat(rfid): read card UIDs without blocking`

3.6 Project cards

Done _____

START STATE: 3.5 passes.

CHECKLIST:

- [ ] Tap each card. Record its UID in the docs.
- [ ] Create a `struct ProjectCard`: UID bytes, UID length, project name, current task.
- [ ] Create a `const` array of your cards.
- [ ] Write a lookup function: compare bytes with `memcmp`, return a pointer or `NULL`.
- [ ] Unknown card prints "Unknown card".
- [ ] Smoke test, then run the Ship checklist.

DONE WHEN: **Each card prints its project. Unknown cards are handled. PR merged.**

COMMIT: `feat(rfid): map card UIDs to projects`

Step 4 Optocoupler bench test

`feature/opto`

GOAL The ESP32 safely presses the PC power button, with no electrical connection between the two.

> **Safety first.** Never wire an ESP32 pin straight to motherboard pins. Inside the PC817, a tiny LED shines on a light sensor, so the signal crosses with no wire between the two circuits. Always work on the PC with the wall plug pulled.

4.1 Pick a drive pin

Done _____

START STATE Step 3 merged. Fresh branch from `main`.

CHECKLIST
- [ ] Review free pins in `PINS.md`. The green/blue RGB LED pins are candidates.
- [ ] Confirm the pin can output and is not a boot strapping pin (a pin the chip reads at power-up).
- [ ] Record the pin and your reason in `wiring.md`. Add it to *Reserved*.

DONE WHEN **Pin chosen and documented.**

COMMIT `docs(wiring): assign optocoupler drive pin`

4.2 Do the resistor math

Done _____

START STATE 4.1 committed. PC817 datasheet downloaded.

CHECKLIST
- [ ] Find in the datasheet: LED forward voltage (Vf), forward current (If), and CTR (current transfer ratio, how much output current you get per input current).
- [ ] Calculate R = (3.3 V − Vf) / If.
- [ ] Check the current is under the ESP32 pin's safe limit.
- [ ] Pick the nearest standard resistor. Recalculate the real current.
- [ ] Write it up in `docs/opto-calc.md` with datasheet page references.

DONE WHEN **Calculation documented and checked against both datasheets.**

COMMIT `docs(hardware): add PC817 resistor calculation`

4.3 Breadboard test

Done _____

START STATE 4.2 committed. Resistor in hand.

CHECKLIST
- [ ] Find PC817 pin 1 (the dot). Pins 1–2 are the LED side, 3–4 the output side.
- [ ] Wire GPIO → resistor → pin 1. Pin 2 → GND.
- [ ] Multimeter in continuity mode across pins 3 and 4.
- [ ] Toggle the pin from a serial command. The meter beeps only while it is on.
- [ ] Photo to `hardware/photos/`.

DONE WHEN **Beeps on command, silent otherwise.**

COMMIT `test(opto): bench-verify PC817 switching`

4.4 Non-blocking press function

Done _____

START STATE: 4.3 passes.

CHECKLIST:

- [ ] `pcPowerPress()` turns the pin on and records the start time.
- [ ] An update function turns it off after about 200 ms. No `delay()`.
- [ ] Ignore new presses while a pulse is active.
- [ ] Trigger it from a serial command.

DONE WHEN: **One command = one clean beep. LED heartbeat never stutters.**

COMMIT: `feat(power): add non-blocking power button pulse`

4.5 Connect to the real PC

Done _____

START STATE: 4.4 passes. PC shut down and unplugged from the wall.

CHECKLIST:

- [ ] Find the front-panel header map in your motherboard manual (PWR_SW pins).
- [ ] Wire the PC817 output in parallel with the case button (a header splitter keeps it clean).
- [ ] Check polarity: emitter (pin 3) to the ground side of PWR_SW (manual or multimeter).
- [ ] Plug back in. Test that the **case button still works first**.
- [ ] Send the command. The PC boots.
- [ ] Document in `wiring.md` with a photo.
- [ ] Smoke test, then run the Ship checklist.

DONE WHEN: **PC boots from your command and the case button still works. PR merged.**

COMMIT: `feat(power): connect optocoupler to PC power header`

Step 5 PC sense

`feature/pc-sense`

GOAL The device knows whether the PC is on.

5.1 Choose a method

Done _____

START STATE Step 4 merged. Fresh branch from `main`.

CHECKLIST
- [ ] List the options: power LED header, a USB port that only has 5 V while the PC is on.
- [ ] For each, note voltage, isolation, wiring effort, and behavior when the PC sleeps.
- [ ] Pick one. Log it in `decisions.md`.

DONE WHEN **Decision and reason committed.**

COMMIT `docs(decisions): choose PC power-sense method`

5.2 Build the sense circuit

Done _____

START STATE 5.1 committed.

CHECKLIST
- [ ] Plan for GPIO 35: input-only with **no internal pull-up**, so it needs an external resistor.
- [ ] Design the circuit. Hint: a second PC817 gives you isolation again.
- [ ] Draw the schematic (a photo of a hand drawing is fine) into `docs/`.
- [ ] Measure the GPIO voltage with the PC on and off. Never above 3.3 V.

DONE WHEN **Circuit built and measured safe in both states.**

COMMIT `docs(hardware): add PC sense circuit`

5.3 pcIsOn() firmware

Done _____

START STATE 5.2 passes.

CHECKLIST
- [ ] Read the pin.
- [ ] Debounce: the reading must stay the same for 50–100 ms before it counts.
- [ ] Log only when the state changes.
- [ ] Test: on → off → sleep.
- [ ] Smoke test, then run the Ship checklist.

DONE WHEN **Serial reports the correct state every time. PR merged.**

COMMIT `feat(power): detect PC on/off state`

Step 6 State machine

`feature/state-machine`

GOAL One clear set of rules decides what the device does at every moment.

6.1 Draw it before coding it

Done _____

START STATE Step 5 merged. Fresh branch from `main`.

CHECKLIST
- [ ] List the states, for example SLEEP, PROMPT, BOOTING, ACTIVE.
- [ ] For each state: what the screen shows and whether the backlight is on.
- [ ] List the events: known card, unknown card, Wake button, Sleep button, timeout, PC on/off.
- [ ] Draw arrows from state to state, each labeled with its event.
- [ ] Write it as a Mermaid diagram in `docs/state-machine.md` (GitHub draws it for you).

DONE WHEN **Diagram committed and every event has an arrow.**

COMMIT `docs(design): add state machine diagram`

6.2 Skeleton

Done _____

START STATE 6.1 committed.

CHECKLIST
- [ ] Create an `enum` of your states.
- [ ] Keep the current state as a `static` variable in a state module.
- [ ] `stateUpdate()` holds a `switch` with one case per state.
- [ ] A `changeState()` helper logs `OLD -> NEW`.
- [ ] Serial commands fake each event for testing.

DONE WHEN **You can force every transition from serial and see it logged.**

COMMIT `feat(state): add state machine skeleton`

6.3 Card wakes the screen

Done _____

START STATE 6.2 passes.

CHECKLIST
- [ ] Add backlight on/off to the display module.
- [ ] Known card in SLEEP → backlight on → PROMPT.
- [ ] No touch for about 30 s in PROMPT → SLEEP.
- [ ] Test the loop: tap, prompt, wait, dark.

DONE WHEN **Tap, prompt, wait, dark all work repeatedly.**

COMMIT `feat(state): wake screen on known card`

6.4 Buttons do real things

Done _____

START STATE: 6.3 passes.

CHECKLIST:

- [ ] Wake Up: if `!pcIsOn()` → `pcPowerPress()` → BOOTING.
- [ ] BOOTING → ACTIVE once `pcIsOn()` is true.
- [ ] Sleep → backlight off → SLEEP.
- [ ] Test both paths on the real PC.

DONE WHEN: **Both paths work on the real PC.**

COMMIT: `feat(state): wire buttons to power actions`

6.5 Edge cases

Done _____

START STATE: 6.4 passes.

CHECKLIST:

- [ ] Unknown card: short message, then back to sleep.
- [ ] Extra taps during BOOTING are ignored.
- [ ] PC already on: skip the press, go straight to ACTIVE.
- [ ] PC never turns on: timeout and show an error.
- [ ] Every case is logged.
- [ ] Smoke test, then run the Ship checklist.

DONE WHEN: **Each edge case behaves sensibly and is logged. PR merged.**

COMMIT: `fix(state): handle unknown card, double tap, and boot timeout`

Step 7 HUD

`feature/hud`

GOAL The glass shows the time, active project, and current task. (HUD = Heads-Up Display.)

7.1 Secrets file

Done _____

START STATE Step 6 merged. Fresh branch from `main`.

CHECKLIST

- [ ] Add `include/secrets.h` to `.gitignore` **before** creating it.
- [ ] Create `secrets.h` with your Wi-Fi name and password.
- [ ] Create `secrets.example.h` with placeholders. This one is committed, so future-you knows the format.
- [ ] `git status`: `secrets.h` is not listed.

DONE WHEN **Real secrets never appear in Git.**

COMMIT `chore(config): add secrets template and ignore rule`

7.2 Real time over the network

Done _____

START STATE 7.1 committed.

CHECKLIST

- [ ] Connect to Wi-Fi without blocking: check the status in `loop()` instead of waiting in a loop.
- [ ] Set up NTP (Network Time Protocol) with the US Central time zone rule so daylight saving is automatic.
- [ ] Print the local time once it syncs.
- [ ] If Wi-Fi fails, the device keeps working and shows `--:--`.

DONE WHEN **Serial shows correct local time. Unplugging the router doesn't crash anything.**

COMMIT `feat(net): sync time over NTP`

7.3 Layout sketch

Done _____

START STATE 7.2 passes.

CHECKLIST

- [ ] Sketch at 320×240: big clock, project, task.
- [ ] Plan for the mirror: bright text on pure black (black keeps the mirror reflective).
- [ ] Save the sketch to `docs/hud-layout.png`.

DONE WHEN **Layout saved.**

COMMIT `docs(design): add HUD layout sketch`

7.4 Clock

Done _____

START STATE: 7.3 committed.

CHECKLIST:
- [ ] Large-font LVGL label.
- [ ] Update it once a second with an `lv_timer`.
- [ ] Compare with your phone for 5 minutes.

DONE WHEN: **Clock matches your phone.**

COMMIT: `feat(hud): add live clock`

7.5 Project and task

Done _____

START STATE: 7.4 passes.

CHECKLIST:
- [ ] Labels read from the active `ProjectCard`.
- [ ] Tapping another card while ACTIVE switches project.
- [ ] Test every card.

DONE WHEN: **Each card changes the HUD.**

COMMIT: `feat(hud): show active project and task`

7.6 Mirror test

Done _____

START STATE: 7.5 passes.

CHECKLIST:
- [ ] Hold the screen behind the mirror in daytime and night lighting.
- [ ] Adjust colors and brightness until it reads well in both.
- [ ] Record the final colors in the docs.
- [ ] Smoke test, then run the Ship checklist.

DONE WHEN: **Readable through the mirror in your normal lighting. PR merged.**

COMMIT: `style(hud): tune colors for two-way mirror`

Step 8 Install and release

`feature/install`

GOAL The device lives on the workbench and runs for days without attention.

8.1 Plan the mount

Done _____

START STATE Step 7 merged. Fresh branch from `main`.

CHECKLIST
- [ ] Measure the mirror, screen, and frame.
- [ ] Test that RFID reads through your frame material and thickness.
- [ ] Sketch with measurements into `hardware/`.
- [ ] Write a BOM (Bill of Materials, the parts list).

DONE WHEN **Mount plan and BOM committed.**

COMMIT `docs(hardware): add mount plan and BOM`

8.2 Power supply

Done _____

START STATE 8.1 committed.

CHECKLIST
- [ ] Remember: the device must stay powered while the PC is **off**, or it can't wake it.
- [ ] Choose the supply. Measure the current draw with a USB power meter.
- [ ] Document the choice and the measured current.

DONE WHEN **Power documented and tested with the PC off.**

COMMIT `docs(hardware): document power supply choice`

8.3 Install and route cables

Done _____

START STATE 8.2 committed.

CHECKLIST
- [ ] Mount everything. Route cables to the PC header.
- [ ] Label both ends of every cable. Add strain relief so a tug can't pull a wire out.
- [ ] Case power button still works.
- [ ] Full functional test: card → wake → PC boots → HUD → sleep.
- [ ] Photos to `hardware/photos/`.

DONE WHEN **Installed and every function works in place.**

COMMIT `docs(hardware): add install photos`

8.4 24-hour soak test

Done _____

START STATE: 8.3 passes.

CHECKLIST:

- [ ] Add uptime and free heap memory to the heartbeat log (a shrinking heap means a memory leak).
- [ ] Log serial output to a file for 24 hours.
- [ ] Check: no resets, heap stays steady.
- [ ] Record the results in `docs/test-log.md`.

DONE WHEN: **24 h with no crashes and a stable heap.**

COMMIT: `test(system): record 24h soak test results`

8.5 Release v1.0.0

Done _____

START STATE: 8.4 passes.

CHECKLIST:

- [ ] README: what it is, photo, how to build, link to wiring.
- [ ] Update `requirements.md` status. Add `CHANGELOG.md` with v1.0.0.
- [ ] Run the Ship checklist.
- [ ] `git tag -a v1.0.0 -m "JARVIS v1"` → `git push origin v1.0.0`.
- [ ] Create a GitHub Release from the tag.

DONE WHEN: **Tag pushed and release published. v1 complete.**

COMMIT: `docs(release): v1.0.0`